Reinforcement Learning for Snake: From Tactical Heuristics to Recurrent Agents

Campigotto Francesco [1]

Abstract

This study proposes a Reinforcement Learning pipeline based on Proximal Policy Optimization (PPO) to solve the game of Snake under fully observable (FO) and partially observable (PO) conditions. For the FO setting, we implement a Convolutional Neural Network (CNN) enhanced by rotation-based data augmentation to learn dynamic spatial management and surpass robust graph-search heuristics. To tackle the non-Markovian PO setting restricted to a 3×3 visual window, we introduce a Recurrent PPO architecture utilizing a Gated Recurrent Unit (GRU) to encode temporal memory. This recurrent memory enables the agent to successfully infer hidden states and track off-screen targets despite severe visibility constraints. Furthermore, we implement a two-stage curriculum learning scaffolding and targeted reward shaping to overcome training instability caused by sparse feedback. Ultimately, these specific architectural and training enhancements allow the agents to navigate strict topological constraints and consistently exceed the performance of systematic exploration baselines.

1. Introduction

The classic game of Snake serves as a rigorous benchmark for sequential decision-making within strictly constrained geometric environments (1). Although the fundamental mechanics rely solely on navigating a grid to consume targets, the underlying control problem becomes increasingly complex over time. As the agent's length increases, the active state space expands exponentially while the available navigable area monotonically decreases. Under these conditions, short-sighted greedy strategies—such as moving directly toward the nearest target—frequently result in self-collisions. Consequently, successful navigation requires sophisticated topological reasoning and long-term trajectory planning to maintain structural connectivity. To investigate the capacity of reinforcement learning algorithms to model these spatial dependencies, this study evaluates agent performance across two distinct observational paradigms:

1. **Fully Observable (FO):** The agent has access to the complete 7×7 grid state.
2. **Partially Observable (PO):** The agent's vision is restricted to a 3×3 window, requiring memory to estimate the state of the world.

To ensure full reproducibility of the results, all random seeds across the environment, NumPy, and TensorFlow were strictly fixed to 0.

2. Critical Issues

Several critical issues inherent to the Snake environment that the models must overcome were identified:

2.1. Sparse & Delayed Rewards

This issue concerns the difficulty of the learning signal itself. While consuming a fruit provides an immediate positive reward, the long sequence of actions leading up to that event yields no reward. This sparsity makes credit assignment difficult, as the agent must associate past moves with a distant reward. Furthermore, negative feedback is often severely delayed; a "bad" move might lead into a "Deadly Corridor" where the terminal collision does not occur until many steps later, making it hard for the agent to identify the original mistake.

2.2. Self-Entrapment

This issue concerns the geometric constraints of the limited 7×7 grid. Space management is paramount; while a greedy approach that minimizes the distance to the fruit is otherwise correct, the essential challenge is topological. The agent must distinguish which of the various greedy paths is safe to ensure future board connectivity. An effective policy must look beyond the immediate reward to avoid movements that inadvertently create closed loops with the agent's own body

[1]University of Padua, Italy. Correspondence to: Campigotto Francesco <francesco.campigotto@studenti.unipd.it>.

Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 2021. Copyright 2021 by the author(s).

segments, which would bifurcate the available space and lead to inevitable failure.

2.3. Partial Observability (PO Specific)

In the PO setting, the agent cannot see its own tail or the fruit if they are outside the 3×3 field of view. This introduces the need for temporal memory. The motivation is not solely to recall the location of a previously sighted fruit, as it may spawn in an unobserved area. Instead, memory is utilized to create a correlation between the history of movements and the current hidden state of the snake's body or the potential position of the fruit, thereby improving decision-making efficacy in the absence of immediate visual targets.

3. Methodology

To systematically address the challenges of the Snake environment across both visibility settings, a unified reinforcement learning pipeline based on Proximal Policy Optimization (PPO) was implemented, tailored with specific architectural and training enhancements.

3.1. Core Algorithm: PPO

Proximal Policy Optimization was selected as the primary optimization algorithm due to its clipped surrogate objective, which limits the probability ratio $r_t(\theta)$ to the range $[1-\epsilon, 1+\epsilon]$. This prevents destructively large policy updates and yields significantly higher training stability compared to standard Policy Gradient methods (2). In highly constrained geometric environments such as Snake, a single suboptimal action can precipitate immediate terminal entrapment, making the learning process highly susceptible to policy collapse. The clipping mechanism mitigates this risk by ensuring constrained, monotonic policy improvements, rendering PPO exceptionally well-suited for navigating this brittle reward landscape. Consequently, PPO constitutes a highly robust choice for this problem class, systematically outperforming both high-variance on-policy methods (e.g., standard A2C) and off-policy value-based methods (e.g., DQN). Furthermore, in the Partially Observable setting where states are non-Markovian, combining PPO's on-policy objective with recurrent memory is widely considered superior. While off-policy recurrent methods (such as DRQN) frequently suffer from severe instability during experience replay due to "stale" hidden states, Recurrent PPO evaluates sequentially valid trajectories purely on-policy, ensuring accurate credit assignment and stable temporal mapping (3).

3.2. Network Architectures

To effectively process the environmental states, two distinct neural network architectures were employed based on the observation paradigm:

1. **Fully Observable (Deep CNN):** The agent receives a full 7×7 grid. The backbone consists of three convolutional layers (32, 64, 64 filters) with 3×3 kernels and `padding='same'` to preserve spatial resolution. A 256-unit dense layer then branches into distinct Actor and Critic heads.

2. **Partially Observable (CNN+GRU):** The agent receives a restricted 3×3 Field of View (FOV). A single convolutional layer with 64 filters and a 3×3 kernel with "valid" padding is used, producing a 1×1 feature map that captures all spatial relationships in the FOV in one operation. Following a 128-unit projection layer, a Gated Recurrent Unit (GRU) with 128 units maintains a hidden state h_t across timesteps. A GRU was chosen over an LSTM for its comparable performance on shorter sequences with fewer parameters (4). This recurrent memory is essential for the agent to encode latent states when targets exit the frame.

3.3. Training Enhancements

Several algorithmic enhancements were developed to accelerate convergence and guide policy discovery:

- **Data Augmentation:** Snake is inherently isotropic. To virtually quadruple the training data and improve generalization, domain-specific augmentations were applied. In the FO setting, observations were explicitly rotated by $0°, 90°, 180°$, or $270°$. In the PO setting, horizontal and vertical flips were applied during rollout collection, with actions inversely transformed for execution, forcing the model to learn reflection-invariant strategies (5).

- **Curriculum Learning (PO Specific):** Direct training on a 7×7 board in the PO setting proved unstable due to sparse rewards. A two-stage curriculum was implemented: Phase 0 trains on a 5×5 board to ensure frequent fruit discovery, acting as a "scaffolding" for basic movement. Phase 1 transfers the weights to the target 7×7 board once the rolling average reward exceeds 0.10 (6).

- **Reward Shaping:** To combat sparse rewards, Manhattan Distance reward shaping was evaluated. During the FO phase, rigorous testing revealed that the inclusion of reward shaping resulted in a slight performance drop of approximately 0.3 points. Instead, drastic degradation in performance was observed with the inclusion of residual connections. Although the negative impact of reward shaping was minor, it is hypothesized that distance-based rewards create deceptive local optima that conflict with the long-term topological safety required to avoid self-entrapment (7); it was therefore discarded to preserve optimal policy discovery. Conversely, in the PO ("blind") environment, reward shaping with a weight of $\alpha = 0.005$

proved beneficial, providing a necessary spatial gradient to guide the snake toward unseen fruit.

4. Experiments and Results

4.1. Phase 1: Fully Observable Environment

4.1.1. Baseline: Tactical Agent

To benchmark the RL agent, a rule-based "Tactical Agent" (`baseline.py`) was developed. This baseline is designed to represent a high-level non-learning solution that leverages graph search and deterministic heuristics. It is sufficiently robust because it ensures that the agent never makes a "suicidal" move if a safe path exists, while actively managing board space. The agent operates through a hierarchical decision process:

1. **Efficiency Check:** The agent first calculates a Breadth-First Search (BFS) path to the fruit, treating all current body segments and walls as static obstacles. It compares this "safe" distance (d_{safe}) with the direct Manhattan distance (d_{direct}). If d_{safe} is significantly larger than d_{direct}, it indicates the body is blocking the optimal route.
2. **Pathfinding with Hugging:** If the path is efficient, the agent evaluates all safe adjacent moves. It prioritizes the shortest BFS path. As a tie-breaker, it counts "hugging" neighbors (adjacent walls or body segments). It prefers moves that keep the snake close to existing obstacles, a tactic designed to keep the center of the 7×7 board open for future maneuvering.
3. **Tactical Reset (Tail-Bite):** If no safe BFS path exists (entrapment) or if the current safe path is highly inefficient, the agent performs a 180° turn—biting its own tail. This reset is only executed if reversing the current direction places the head closer to the fruit than the inefficient safe path.

Despite its robustness, a fundamental limitation is that the baseline treats the current body segments as static obstacles. It does not take into account the complex future motion or displacement of the snake's body segments as the game progresses, which can lead to suboptimal positioning in long-duration episodes.

4.1.2. Hyperparameters

- **Rollout ($N = 20$) & Clipping:** Provides 10,000 transitions per update to stabilize gradient estimates. Policy updates are constrained by a PPO clip range of $\epsilon = 0.2$, and gradient clipping is set to 0.5.
- **Discount & GAE:** $\gamma = 0.99$ ensures long-term survival, while $\lambda = 0.95$ balances the bias-variance trade-off.
- **Linear Scheduling:** Learning rate ($2e^{-4} \rightarrow 5e^{-5}$) and entropy ($0.02 \rightarrow 0.005$) decay over 5000 updates to shift smoothly from exploration to exploitation.

4.1.3. Results

The final model demonstrates exceptional performance, successfully solving the complex dependencies of the environment.

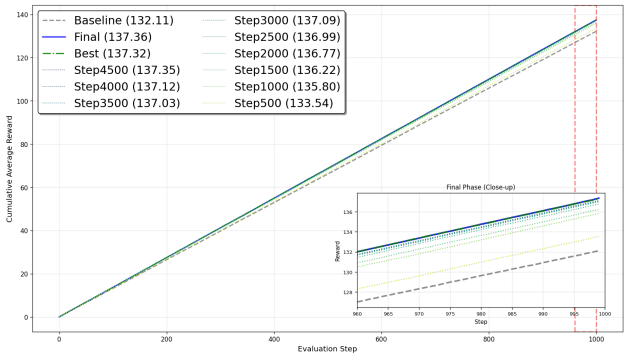

Figure 1. Fully Observable (FO): Cumulative reward comparison evaluated over 1000 steps on 500 parallel environments. The plot displays the Tactical Baseline alongside the RL agent's final model, the "best" model, and intermediate checkpoints saved every 500 updates.

1. **Performance vs. Baseline:** As illustrated in Figure 1, the PPO agent achieves a significantly higher cumulative reward compared to the heuristic baseline when evaluated over a rigorous 1000 steps on 500 parallel environments.
   - **Baseline Score:** ≈ 132 average reward.
   - **RL Final Score:** ≈ 137 average reward.

   This difference of ≈ 5 points is substantial in the context of Snake. Notably, after just 500 updates, the model already outperformed the baseline. In the last 2500 updates, the model improved by only ≈ 0.4 points, indicating near-convergence. Therefore, using more than 5000 updates does not seem necessary considering the already long training time of the model.
2. **Training Stability:** Figure 2 demonstrates robust training stability. The combination of rotation-based augmentation and parameter scheduling allowed the agent to learn rapidly and steadily refine its policy.
3. **Weight Selection:** For final evaluation, parameters of the model achieving the best average reward during training were also evaluated, demonstrating that peak training performance does not always guarantee the best evaluation results.

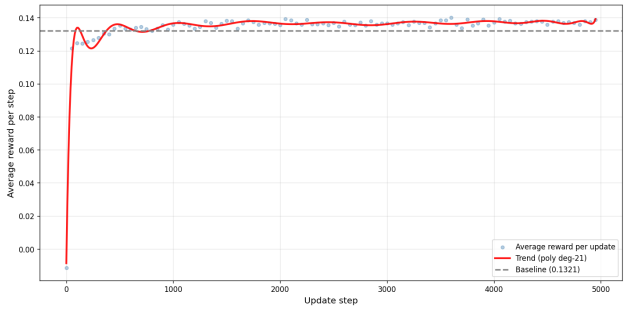

Figure 2. Fully Observable (FO): Training progress. The agent converges to a policy that consistently yields high rewards.

4.1.4. Comparative Analysis: New vs. Old Approach

A comparative analysis was conducted between the final model and an initial iteration developed during the early stages of this project.

Performance Impact: The previous approach was only able to meet the baseline standard, effectively converging to a conservative and sub-optimal policy. By contrast, the implementation of batch reuse, 4-fold rotational augmentation, and parameter scheduling transformed the agent, allowing it to break the performance ceiling and improve a lot over the baseline. The resulting +5 point margin represents a qualitative leap in handling edge cases where the heuristic typically fails.

Computational Cost: This performance gain comes at a significant computational cost.

- **Training Time:** The new approach is considerably slower. The use of $K = 4$ epochs means the network is updated four times for every batch of data, compared to a single pass in the initial approach.
- **Overhead:** The explicit rotation of 500 boards at every timestep during rollout adds a non-trivial CPU overhead.

In summary, while the new training pipeline is computationally more expensive and time-consuming, it was necessary to achieve super-human (or super-heuristic) performance in this environment.

4.2. Phase 2: Partially Observable Environment

4.2.1. Baseline: Square Explorer

The heuristic baseline for the partially observable task (`baseline_exploration.py`) is designed to solve the exploration-exploitation trade-off when the fruit is not immediately visible. This baseline is sufficiently good because it provides a systematic way to sweep the board while maintaining a high degree of survival. The agent switches between two primary modes:

1. **HUNT Mode:** If a fruit is detected within the 3×3 FOV, the agent converts FOV coordinates to board coordinates and attempts to find a valid BFS path. It prioritizes moves that stay closer to corners and hug walls to maintain structural safety.
2. **PATROL Mode:** If no fruit is visible, the agent follows a systematic patrol path around the inner perimeter of the board ($r, c \in [2, 4]$). This "Square Explorer" logic ensures the agent covers the maximum possible area with its limited FOV to probabilistically find newly spawned fruits.
3. **Center Avoidance:** The agent is explicitly programmed to avoid the center cell (3, 3) unless a fruit is present there. This prevents the agent from being easily surrounded or cornered in the most open part of the grid.

A critical limitation of this baseline is that the patrol path is hardcoded; consequently, a new predetermined path or manual adjustment of the heuristic would be required if the dimensions of the board or the Field of View (FOV) were to change.

4.2.2. Hyperparameters

The PO agent utilized Truncated Backpropagation Through Time (BPTT) (8).

- **Rollout ($N = 25$) & Mini-batches:** 500 parallel environments sliced by board (50 per batch) to preserve hidden state continuity for BPTT.
- **Clipping & Regularization:** Policy updates are constrained by a PPO clip range of $\epsilon = 0.2$, and gradient clipping is set to 0.5. $\gamma = 0.99$ and GAE $\lambda = 0.95$ stabilize the recurrent updates.
- **Linear Scheduling:** Learning rate ($2{\cdot}10^{-4} \rightarrow 5{\cdot}10^{-5}$) and entropy ($0.04 \rightarrow 0.005$) were linearly decayed over the 7500 updates for fine-tuned mapping.

4.2.3. Results

The Recurrent PPO agent demonstrates strong adaptation to limited vision, though the training profile reveals distinct behaviors compared to the FO setting.

1. **Performance vs. Baseline:** As illustrated in Figure 3, the final recurrent agent successfully surpassed the heuristic baseline after 7500 updates.
   - **Baseline Score:** ≈ 98.5 average reward.
   - **RL Final Score:** ≈ 99.4 average reward.

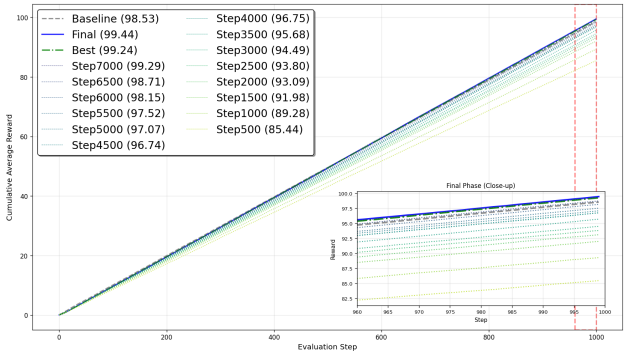

Figure 3. Partially Observable (PO): Cumulative reward comparison evaluated over 1000 steps on 500 parallel environments. The plot displays the Tactical Baseline alongside the RL agent's final model, the "best" model, and intermediate checkpoints saved every 500 updates.

The agent overcame the baseline only in the final 1000 steps, improving by ≈ 0.7 points. The upward trend suggests training has not entirely converged, but computational constraints (over half a day of training) prevented further updates.

2. **Training Stability:** Figure 4 highlights training stability. Every 500 steps, evaluation results were equal or better, confirming an effective method. The curriculum transition from the 5×5 domain to the 7×7 domain correlates with a significant jump in performance, proving the scaffolding approach crucial for stability. Even if the trend curve do not surpass the baseline, the final model perform better.

3. **Weight Selection:** As before, the performance of the parameters that achieved the best average points during training across the 500 boards was calculated. This "best" model performed slightly worse than the final model. This reiterates the finding that even with a large sample size of 500 parallel environments, high performances during training do not always correspond to good final results during rigorous evaluation.

4.2.4. Comparative Analysis: New vs. Old Approach

A comparative analysis was conducted between the final model and an initial, much simpler iteration developed during the early stages of this project. The previous approach utilized a basic PPO configuration that lacked the advanced training features of the final pipeline, such as Curriculum Learning, Symmetric Data Augmentation, and Reward Shaping.

Performance Impact: The basic approach achieved an average reward of only ≈ 86 points, which is significantly lower than both the heuristic baseline and the final Recurrent PPO agent. Furthermore, this simpler model converged prematurely after just 5000 updates, demonstrating an inability to further learn or adapt to the complexities of the partially observable environment. By contrast, the implementation of the advanced features allowed the final agent to break this performance ceiling and continue improving beyond 7500 updates.

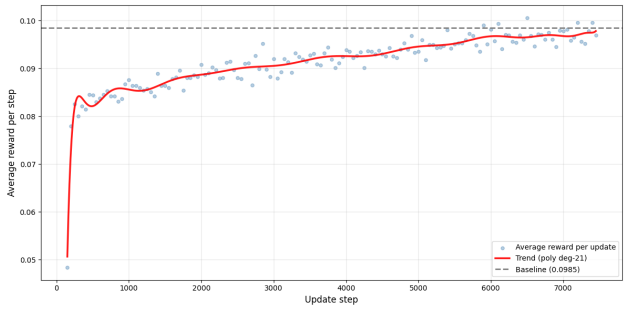

Figure 4. Partially Observable (PO): Training progress showing the learning trend after the curriculum change.

Computational Cost: As in the fully observable phase, this performance gain comes at a computational cost, requiring longer training times and more complex rollout data generation. However, it was strictly necessary to achieve competitive performance in this challenging "blind" environment.

5. Discussion

The primary objective of this study was to evaluate whether RL agents could surpass the performance of heuristic baselines. It is important to note that the baseline strategies were defined and tested through manual human gameplay. Therefore, they represent logical, common-sense approaches utilized by a normal human, rather than mathematically perfect or optimal strategies.

Fully Observable Environment: In the FO setting, the superior performance of the PPO agent was expected. As discussed in Section 4.1.1, the tactical baseline treats the snake's body as a set of static obstacles, failing to account for the future movement of the tail which periodically frees up navigable space. The RL agent successfully learned to anticipate these dynamic topological changes, identifying more optimal, forward-looking paths. Because of this structural limitation in the heuristic, even simpler early RL iterations were able to achieve scores very close to the baseline.

Partially Observable Environment: Conversely, in the PO

setting, it was not a given that the RL agent would overcome the baseline. Operating under strict visibility constraints makes reliable navigation exceptionally difficult. While the "Square Explorer" baseline strategy was obviously not optimal, attempting to hand-craft more complex heuristics consistently yielded worse results. Furthermore, alternative RL methods failed to even slightly match the baseline's performance. The Recurrent PPO model ultimately succeeded in finding a strategy robust enough to beat the baseline, though it required extensive training to do so.

Computational Constraints: As noted in the PO results, the recurrent model did not fully converge even after half a day of training. The most computationally heavy components of the pipeline are the large N-step rollout generation and the synchronization of the parallel boards. However, empirical testing confirmed that these specific parameters were the optimal values to achieve the best peak performance alongside the best training consistency.

Future Improvements: Resolving Windows-specific configuration issues to fully migrate the training pipeline to a GPU would drastically reduce training durations. Additionally, minor performance improvements might be achieved by exploring different Data Augmentation techniques or by perfecting the Reward Shaping mechanics previously discarded in the Fully Observable environment, which could provide a denser learning signal without disrupting the optimal policy. Finally, replacing the hard-coded patrol path in the partially observable setting with a dynamic coverage algorithm would improve benchmark flexibility across varying board sizes.

6. Conclusion

This study successfully demonstrated the efficacy of Reinforcement Learning in solving the highly constrained spatial problem of Snake. In the Fully Observable environment, a CNN-based PPO agent rapidly surpassed robust human-crafted heuristics by learning dynamic, forward-looking spatial management. In the more complex Partially Observable environment, where standard methods fail, the integration of Recurrent PPO with Curriculum Learning proved essential. By effectively encoding temporal memory, the recurrent agent successfully navigated blind states and ultimately exceeded the baseline strategy, proving the capacity of advanced RL architectures to discover superior, generalized policies in environments with sparse rewards and severe geometric constraints.

References

[1] Pan, Y., Song, Y., Ma, Q., Gu, B., Dong, J., & Tang, Z. (2023). Playing the Snake Game with Reinforcement Learning. *Cambridge Explorations in Arts and Sciences*, pp. 1–2. https://doi.org/10.61603/ceas.v1i1.13

[2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, pp. 5–8. https://arxiv.org/abs/1707.06347

[3] Hausknecht, M., & Stone, P. (2015). Deep Recurrent Q-Learning for Partially Observable MDPs. In *2015 AAAI Fall Symposium Series*, pp. 5–6. https://arxiv.org/abs/1507.06527

[4] Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. In *NIPS 2014 Workshop on Deep Learning*, pp. 7–8. https://arxiv.org/abs/1412.3555

[5] Laskin, M., Lee, K., Stooke, A., Pinto, L., Abbeel, P., & Srinivas, A. (2020). Reinforcement Learning with Augmented Data. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 9. https://arxiv.org/abs/2004.14990

[6] Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum Learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pp. 8. https://doi.org/10.1145/1553374.1553380

[7] Mannion, P., Devlin, S., Mason, K., & Howley, E. (2017). Policy Invariance under Reward Transformations for Multi-Objective Reinforcement Learning. *Neurocomputing*, pp. 8–13. https://doi.org/10.1016/j.neucom.2017.05.090

[8] Aicher, C., Foti, N. J., & Fox, E. B. (2020). Adaptively Truncating Backpropagation Through Time to Control Gradient Bias. In *Proceedings of the 35th Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 7–9. http://proceedings.mlr.press/v115/aicher20a.html